

Defend-The-Core

Annexe 1 : Réseau & OPNsense

Pare-feu / routeur inter-VLAN — zoning, filtrage default-deny, NAT et intégration active response

[Segmentation ANSSI](#) · [Default-deny](#) · [API REST](#) · [Infrastructure as Code](#)

Table des matières

1. Zoning et segmentation VLAN	3
1.1. Niveaux de confiance	3
1.2. Règle d'or : default-deny	3
2. Topologie VirtualBox	4
2.1. Plan d'adressage	4
2.2. Affectation des hôtes	4
3. Configuration OPNsense	5
3.1. Activation de l'API REST	5
3.2. Scripts d'automatisation	5
3.3. Alias réseau	6
4. Règles de filtrage	6
4.1. Ordre d'évaluation	7
4.2. Logging	7
5. NAT et port forwarding	8
5.1. NAT sortant	8
5.2. Port forwarding (services publics)	8
5.3. Alias dynamique et active response	8
6. Choix OPNsense vs pfSense	9
7. Intégration Wazuh (active response)	9
7.1. Mécanisme de blocage automatique	10
7.2. Règle de pare-feu correspondante	10
7.3. Sécurités du script active response	10

1. Zoning et segmentation VLAN

Dans un réseau « plat », la compromission d'un seul poste utilisateur suffit à un attaquant pour scanner l'ensemble du système d'information, identifier les serveurs et exploiter leurs vulnérabilités. C'est le schéma classique de la propagation latérale : phishing → compromission poste → reconnaissance → mouvement latéral → exfiltration. La **segmentation en zones de confiance** (zoning) brise cette chaîne : même si un poste est compromis, l'attaquant ne voit et ne peut atteindre que ce qui lui est explicitement autorisé.

L'infrastructure Defend-The-Core est découpée en **quatre zones de confiance**, chacune matérialisée par un VLAN distinct et cloisonnée par un pare-feu OPNsense en **default-deny**. Le niveau de confiance décroît du VLAN 99 (administration) au VLAN 10 (bureautique, considéré comme non fiable car exposé au vecteur phishing).

Tableau 1 — Zones de confiance et segmentation VLAN

VLAN	Zone	Réseau	Confiance	Rôle
10	Bureautique	10.10.10.0/24	Faible	Postes utilisateurs (vecteur d'attaque n°1)
20	DMZ	10.10.20.0/24	Moyenne	Services exposés : reverse proxy, passerelle VPN
30	Critique	10.10.30.0/24	Haute	Serveurs métier (Web + BDD), Active Directory
99	Admin / SIEM	10.10.99.0/24	Maximale	Surveillance (Wazuh), bastion d'administration (NixOS)

1.1. Niveaux de confiance

Chaque zone se voit attribuer un niveau de confiance reflétant son exposition et la sensibilité des données qu'elle héberge. Ce niveau détermine la matrice des flux autorisés : un flux ne peut aller que d'une confiance **égale ou supérieure** vers une confiance **égale ou inférieure**, jamais l'inverse. Concrètement, le VLAN 99 (admin) peut initier des flux vers le VLAN 30 (critique) pour l'administration, mais le VLAN 10 (bureautique) ne peut jamais initier de flux vers le VLAN 30 ni le VLAN 99.

1.2. Règle d'or : default-deny

Le pare-feu OPNsense applique une politique **default-deny** : tout flux non explicitement autorisé est rejeté. Chaque interface possède une règle finale implicite *block any any*. La philosophie ANSSI du cloisonnement impose qu'un flux ne puisse traverser une zone que s'il est **nécessaire, identifié et tracé**.

Règle d'or : un flux ne peut aller que d'une confiance égale ou supérieure vers une égale ou inférieure, et uniquement sur des ports métier explicites. Le retour vers le VLAN 10 n'est **jamais** initié depuis une zone plus sensible.

Ce principe transforme un compromission de poste en incident contenu : l'attaquant bloqué sur le VLAN 10 ne peut ni atteindre les bases de données (VLAN 30), ni compromettre le SIEM ou le bastion (VLAN 99). La latéralité est structurellement impossible.

2. Topologie VirtualBox

L'infrastructure est déployée sous VirtualBox. Chaque VLAN correspond à un **réseau interne VirtualBox** distinct (Internal Network), ce qui simule le zoning physique / VLAN 802.1Q sans équipement actif. OPNsense possède **5 interfaces réseau** : 1 NAT (WAN, accès Internet) + 4 réseaux internes correspondant aux VLANs 10, 20, 30 et 99. Le pare-feu est ainsi le seul point de passage entre les zones, garantissant que **aucun flux ne contourne le filtrage**.

Tableau 2 — Mapping interfaces OPNsense / réseaux internes VirtualBox

Interface	Réseau interne VirtualBox	VLAN	OPNsense	Adressage
vtnet0	vbox-wan (NAT)	—	WAN	DHCP (10.0.2.0/24)
vtnet1	vbox-vlan10	10	LAN	10.10.10.1/24
vtnet2	vbox-vlan20	20	OPT1	10.10.20.1/24
vtnet3	vbox-vlan30	30	OPT2	10.10.30.1/24
vtnet4	vbox-vlan99	99	OPT3	10.10.99.1/24

2.1. Plan d'adressage

Le référentiel d'adressage est 10.10.X.0/24 où X identifie le VLAN. OPNsense porte toujours l'IP .1 de chaque sous-réseau et sert de passerelle par défaut. Aucun hôte ne possède de route statique vers un autre VLAN : tout transite par OPNsense, qui filtre chaque flux.

Tableau 3 — Plan d'adressage par VLAN

VLAN	Réseau	Passerelle	Masque	Plage hôtes	DHCP
10	10.10.10.0/24	10.10.10.1	255.255.255.0	.10 – .200	Oui (OPNsense)
20	10.10.20.0/24	10.10.20.1	255.255.255.0	.10 – .100	Non (statique)
30	10.10.30.0/24	10.10.30.1	255.255.255.0	.10 – .100	AD (Win Server)
99	10.10.99.0/24	10.10.99.1	255.255.255.0	.10 – .50	Non (statique)

Le DHCP n'est activé que sur le VLAN 10 (bureautique). Les zones sensibles (DMZ, admin) fonctionnent en adresses strictement statiques, conformément au principe du moindre privilège. Le VLAN 30 délègue son DHCP au contrôleur Active Directory.

2.2. Affectation des hôtes

Hôte	VLAN	IP	Rôle	OS
OPNsense	tous	.1 / VLAN	Pare-feu / routeur	OPNsense
Win10 (employé)	10	10.10.10.50	Poste utilisateur	Windows 10

Kali (attaquant)	10	10.10.10.100	Simulation d'attaque	Kali Linux
Reverse proxy	20	10.10.20.10	Proxy inverse (DMZ)	Ubuntu 22.04
Passerelle VPN	20	10.10.20.20	Accès distant	Ubuntu 22.04
Ubuntu (Web + BDD)	30	10.10.30.10	Serveur métier	Ubuntu 22.04
Windows Server (AD)	30	10.10.30.20	AD / DNS / DHCP	Windows Server 2022
Wazuh (SIEM)	99	10.10.99.10	SIEM / XDR + Elastic	Ubuntu 22.04
Bastion NixOS	99	10.10.99.20	PAW / administration	NixOS

3. Configuration OPNsense

OPNsense est configuré de façon **reproductible** via une API REST, pilotée par quatre scripts shell versionnés dans le dépôt (opnsense/scripts/). Cette approche Infrastructure as Code garantit que la configuration du pare-feu est auditable, réversible et rejouable — un prérequis pour une posture DevSecOps.

3.1. Activation de l'API REST

Les scripts exploitent l'**API REST native d'OPNsense** (authentification HTTP Basic, clé + secret). Son activation est un préalable indispensable :

- 1. Interface web → **System** → **Settings** → **Administration** — cocher *Enable API*.
- 2. Créer une clé API + secret via **System** → **Access** → **Users** → **root** → **Edit** (onglet « API keys »).
- 3. Renseigner les valeurs dans `.env` :

```
OPNSENSE_API_KEY="..." # clé API générée
OPNSENSE_API_SECRET="..." # secret associé
OPNSENSE_HOST="10.10.99.1" # IP d'administration (VLAN 99)
```

Aucune clé n'est versionnée : le fichier `.env` est exclu du dépôt via `.gitignore`. Seul le modèle `.env.example` est commité.

3.2. Scripts d'automatisation

La configuration est appliquée séquentiellement par quatre scripts, chacun responsable d'un domaine. Ils sont **idempotents** et peuvent être ré-exécutés sans effet de bord.

Tableau 4 — Scripts de configuration OPNsense

Script	Description
00-initial-setup.sh	Configuration initiale et hardening global : vérification de l'API, changement du mot de passe admin, activation HTTPS (port 8443), désactivation des services inutiles (UPnP), syslog distant vers Wazuh, timeout de session, synchronisation NTP.
01-vlan-interfaces.sh	Affectation des adresses IP des interfaces conformément au plan d'adressage (10.10.X.0/24). Activation du DHCP sur le VLAN 10, désactivation explicite sur les VLANs 20/30/99. Rechargement des interfaces (idempotent).

02-firewall-rules.sh	Application de la matrice de filtrage default-deny (voir section 4). Création des alias réseau, puis déploiement des règles block prioritaires et des règles allow métier, avant un block final explicite sur chaque interface.
03-nat-gateway.sh	NAT sortant (masquerade) pour les VLANs autorisés, port forwarding des services publics (443 → reverse proxy, 51820 → VPN), création de l'alias dynamique active response et hardening WAN (bogon, réseaux privés).

Ordre d'exécution (depuis la machine hôte ou le bastion) :

```
export $(cat ../.env | xargs)
bash scripts/00-initial-setup.sh
bash scripts/01-vlan-interfaces.sh
bash scripts/02-firewall-rules.sh
bash scripts/03-nat-gateway.sh
```

3.3. Alias réseau

Les alias centralisent les objets réseau (sous-réseaux et hôtes) pour produire des règles **lisibles et maintenables**. Plutôt que d'écrire `10.10.99.10` dans chaque règle, on référence l'alias `host_wazuh`. Le script `02-firewall-rules.sh` définit **11 alias** réseau (4 sous-réseaux + 7 hôtes), auxquels s'ajoute l'alias dynamique `wazuh_blocked_ips` créé par `03-nat-gateway.sh` pour l'active response.

Tableau 5 — Alias réseau OPNsense (11 alias)

Alias	Valeur	Type	Usage
net_vlan10	10.10.10.0/24	Sous-réseau	Zone bureautique
net_vlan20	10.10.20.0/24	Sous-réseau	Zone DMZ
net_vlan30	10.10.30.0/24	Sous-réseau	Zone critique
net_vlan99	10.10.99.0/24	Sous-réseau	Zone admin / SIEM
host_wazuh	10.10.99.10	Hôte	SIEM Wazuh (destination logs)
host_bastion	10.10.99.20	Hôte	Bastion NixOS (SSH admin)
host_reverse_proxy	10.10.20.10	Hôte	Reverse proxy (DMZ)
host_vpn	10.10.20.20	Hôte	Passerelle VPN (DMZ)
host_ubuntu_web	10.10.30.10	Hôte	Serveur Web + BDD
host_win_server	10.10.30.20	Hôte	AD / DNS / DHCP
host_win10	10.10.10.50	Hôte	Poste utilisateur

4. Règles de filtrage

La matrice de filtrage traduit la politique default-deny en règles explicites. Les règles **block** d'isolation des zones sensibles sont placées en priorité haute, suivies des règles **allow** métier, puis d'un block final explicite sur chaque interface. L'extrait ci-dessous présente les règles principales (le dépôt contient la matrice complète dans `architecture/firewall-rules.md`).

Tableau 6 — Matrice de filtrage default-deny (extrait)

Source	Destination	Port(s)	Action	Justification
VLAN 10	host_reverse_proxy	80, 443	Allow	Accès au reverse proxy uniquement
VLAN 10	host_vpn	443	Allow	Accès à la passerelle VPN
VLAN 10	VLAN 30	any	Block	Isolation totale de la zone critique
VLAN 10	VLAN 99	any	Block	Isolation totale de la zone admin
VLAN 10	WAN	53, 80, 443	Allow	DNS + HTTP(S) sortant (via OPNsense)
VLAN 10	host_wazuh	514	Allow	Syslog bureautique vers Wazuh
Reverse proxy	host_ubuntu_web	80, 443	Allow	Reverse proxy → serveur Web métier
VPN (DMZ)	VLAN 30	any	Block	Le VPN ne doit pas atteindre la BDD directement
VLAN 30	host_win_server	53	Allow	Résolution DNS via l'Active Directory
VLAN 30	host_wazuh	1514, 1515	Allow	Flux de logs des serveurs vers Wazuh
VLAN 30	VLAN 99	any	Block	Aucun autre flux vers la zone admin
Bastion (VLAN 99)	VLAN 30	22	Allow	SSH admin du bastion vers les serveurs
VLAN 99 (hors bastion)	VLAN 30	any	Block	Seul le bastion peut SSH, pas Wazuh
VLAN 99	WAN	123	Allow	NTP pour la synchronisation SIEM
VLAN 99	WAN	80, 443	Allow	Mises à jour Wazuh / Elastic
WAN	host_reverse_proxy	443	Allow	Accès public au reverse proxy
WAN	host_vpn	51820	Allow	Accès VPN entrant (WireGuard)
WAN	VLAN 30	any	Block	Aucune exposition de la zone critique
WAN	VLAN 99	any	Block	Aucune exposition de la zone admin
* (toutes)	any	any	Block	Règle finale implicite — default-deny

4.1. Ordre d'évaluation

Les règles sont évaluées **de haut en bas**, la première correspondance l'emportant (first-match wins). L'ordre reflète la priorité dans OPNsense : les règles **block** d'isolation précèdent les règles **allow**, de sorte qu'un flux interdit ne puisse jamais être autorisé par une règle plus permissive placée plus bas. La règle finale *block any* (présente sur chaque interface) garantit qu'aucun flux oublié ne passe.

4.2. Logging

Toutes les règles **block** et les règles **allow** sensibles (inter-zone) activent explicitement le logging (`log = yes`). Les logs OPNsense ont une double destination :

- consultation locale — *Firewall* → *Log Files* sur l'interface web ;
- transfert vers Wazuh en syslog UDP 514, pour corrélation SIEM en temps réel.

Cette journalisation croisée permet au SIEM de détecter des patterns d'attaque (scans, tentatives répétées) à partir des drops du pare-feu, et de déclencher une active response (voir section 7).

5. NAT et port forwarding

OPNsense assure deux fonctions de traduction d'adresses : le **NAT sortant** (masquerade) qui permet aux VMs autorisées d'accéder à Internet, et le **port forwarding** qui publie les services publics de la DMZ. Ces deux mécanismes sont configurés par le script `03-nat-gateway.sh`.

5.1. NAT sortant

Le NAT sortant fonctionne en mode **hybrid** avec des règles explicites par VLAN. L'accès Internet est accordé aux VLANs 10, 20 et 99, mais **refusé au VLAN 30** (zone critique) : les serveurs métier n'ont aucun besoin d'accéder directement à Internet, et cette absence de route sortante supprime une classe entière de vecteurs d'exfiltration et de C2 (command & control).

Tableau 7 — NAT sortant par VLAN

VLAN	NAT sortant	Justification
10 — Bureautique	Oui (masquerade WAN)	DNS et navigation Web des postes
20 — DMZ	Oui (masquerade WAN)	Mises à jour des services DMZ
30 — Critique	Non (aucune règle)	Isolation sortante — anti-exfiltration
99 — Admin / SIEM	Oui (masquerade WAN)	Mises à jour Wazuh / Elastic, NTP

5.2. Port forwarding (services publics)

Seuls deux services de la DMZ sont publiés vers Internet. Aucun port n'est jamais directement redirigé vers le VLAN 30 ou le VLAN 99, conformément au default-deny.

Tableau 8 — Règles de port forwarding WAN → DMZ

Port WAN	Protocole	Cible (DMZ)	Port local	Service
443	TCP	host_reverse_proxy (10.10.20.10)	443	Reverse proxy public (HTTPS)
51820	UDP	host_vpn (10.10.20.20)	51820	VPN WireGuard (accès distant)

Le reverse proxy sert de point d'entrée unique pour les services Web publics et termine les connexions TLS ; il forward ensuite vers le serveur métier du VLAN 30 sur les ports 80/443 (règle allow dédiée). La passerelle VPN expose le port WireGuard 51820/udp pour l'accès distant.

5.3. Alias dynamique et active response

Le script `03-nat-gateway.sh` crée un alias dynamique `wazuh_blocked_ips` (type host, initialement vide) et une règle **block** placée en priorité haute sur le WAN, dont la source référence cet alias. Wazuh peut ainsi injecter à la volée des IP à bloquer via l'API OPNsense (`/api/firewall/alias/addItem`) ; le rechargement de l'alias propage immédiatement le blocage sur toutes les interfaces. Ce mécanisme est détaillé section 7.

Le script applique également un **hardening WAN** : filtrage des réseaux bogon et des plages privées sur l'interface externe (anti-spoofing).

6. Choix OPNsense vs pfSense

OPNsense et pfSense sont deux pare-feux open-source dérivés de m0n0wall, tous deux basés sur FreeBSD et le moteur pf. Le projet Defend-The-Core a retenu **OPNsense** pour quatre raisons techniques déterminantes, résumées dans le tableau ci-dessous.

Tableau 9 — Comparatif OPNsense vs pfSense

Critère	OPNsense	pfSense
API REST native	API REST intégrée et documentée — automatisation IaC directe	API REST partielle ; dépend de l'API legacy XMLRPC
Interface web	Interface moderne (framework Phalcon), UX claire et responsive	Interface plus ancienne, moins uniforme
Fréquence de mises à jour	Releases majeures fréquentes (~2 semestrielles) + correctifs réguliers	Cycle plus lent, correctifs moins fréquents
Communauté & écosystème	Communauté active, plugins tiers, intégration cloud native	Communauté établie mais plus conservatrice
Licence & modèle	BSD, projet communautaire soutenu par Deciso (pas de verrou)	Apache 2.0, soutenu par Netgate (édition CE vs plus)
Sécurité par défaut	Hardening intégré, reporting IDS/IPS (Suricata) mature	Fonctionnalité IDS/IPS présente, configuration plus manuelle

Verdict : la présence d'une **API REST native** est le critère décisif. Elle rend la configuration du pare-feu entièrement scriptable et reproductible — un prérequis pour la posture Infrastructure as Code du projet. C'est précisément cette API qui permet aux quatre scripts de configurer le zoning, les règles et le NAT sans intervention manuelle, et à Wazuh d'injecter des blocages dynamiques en active response.

7. Intégration Wazuh (active response)

L'active response ferme la boucle détection → réaction. Lorsque Wazuh corrèle une attaque (bruteforce SSH, scan réseau, mouvement latéral) et déclenche une alerte de niveau ≥ 10 , il exécute automatiquement le script `wazuh/rules/active-response/block-attacker.sh`. Ce script ajoute l'IP source à l'alias `wazuh_blocked_ips` sur OPNsense via l'API REST, propageant un blocage immédiat sur l'ensemble des interfaces.

7.1. Mécanisme de blocage automatique

Le flux d'active response se déroule en cinq étapes :

1. **Détection** — Wazuh corrèle les logs (OPNsense syslog, agents) et déclenche une alerte de niveau ≥ 10 (règles 100100, 100101, 100200...).
2. **Déclenchement** — le manager exécute `block-attacker.sh` avec l'IP source de l'alerte.
3. **Injection** — le script valide l'IP (anti-injection) puis l'ajoute à l'alias `wazuh_blocked_ips` via `POST /api/firewall/alias/addItem`.
4. **Propagation** — rechargement de l'alias et du filtre (`alias/reconfigure, filter/apply`) ; le blocage est effectif en quelques secondes.
5. **Débloquage** — après un timeout (3600 s par défaut), Wazuh rappelle le script en mode `delete` et retire l'IP de l'alias.

L'alias `wazuh_blocked_ips` est donc un **pont dynamique** entre le SIEM et le pare-feu : il transforme une détection logicielle en action réseau, sans intervention humaine.

7.2. Règle de pare-feu correspondante

La règle de blocage est créée par `03-nat-gateway.sh` et placée en **priorité haute sur le WAN**, de sorte qu'une IP bloquée soit rejetée avant toute autre règle `allow` :

```
{
  "rule": {
    "interface": "wan",
    "action": "block",
    "protocol": "any",
    "source": "wazuh_blocked_ips",
    "destination": "any",
    "descr": "Block IPs detectees par Wazuh (active response)",
    "log": "1"
  }
}
```

7.3. Sécurités du script active response

Le script `block-attacker.sh` intègre plusieurs garde-fous pour éviter les blocages abusifs ou l'injection :

- **Validation d'IP** — l'IP source est contrôlée par expression régulière avant tout appel API (anti-injection de payload).
- **Whitelist implicite** — le bastion (10.10.99.20), Wazuh (10.10.99.10) et localhost ne sont jamais bloqués, même s'ils apparaissent comme source d'une alerte.
- **Fallback local** — si l'API OPNsense est injoignable ou les clés absentes, le script bascule sur un blocage `iptables` local pour ne jamais rester sans défense.
- **Timeout de déblocage** — configuration 3600 dans `ossec.conf` ; l'IP est libérée automatiquement après 1 h.

Ce mécanisme illustre le pilier « Détection » du projet : un réseau fermé (default-deny) ne suffit pas — il faut aussi **réagir en temps réel** aux attaques observées, sans attendre une intervention humaine.